JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.spi

Class LocaleServiceProvider

java.lang.Object
    java.util.spi.LocaleServiceProvider

Direct Known Subclasses:

BreakIteratorProvider, CalendarDataProvider, CalendarNameProvider, CollatorProvider, CurrencyNameProvider, DateFormatProvider, DateFormatSymbolsProvider, DecimalFormatSymbolsProvider, LocaleNameProvider, NumberFormatProvider, TimeZoneNameProvider

---

```
public abstract class LocaleServiceProvider
extends Object
```

This is the super class of all the locale sensitive service provider interfaces (SPIs).

Locale sensitive service provider interfaces are interfaces that correspond to locale sensitive classes in the `java.text` and `java.util` packages. The interfaces enable the construction of locale sensitive objects and the retrieval of localized names for these packages. Locale sensitive factory methods and methods for name retrieval in the `java.text` and `java.util` packages use implementations of the provider interfaces to offer support for locales beyond the set of locales supported by the Java runtime environment itself.

Packaging of Locale Sensitive Service Provider Implementations

Implementations of these locale sensitive services are packaged using the Java Extension Mechanism as installed extensions. A provider identifies itself with a provider-configuration file in the resource directory META-INF/services, using the fully qualified provider interface class name as the file name. The file should contain a list of fully-qualified concrete provider class names, one per line. A line is terminated by any one of a line feed ('\n'), a carriage return ('\r'), or a carriage return followed immediately by a line feed. Space and tab characters surrounding each name, as well as blank lines, are ignored. The comment character is '#' ('#');

on each line all characters following the first comment character are ignored. The file must be encoded in UTF-8.

If a particular concrete provider class is named in more than one configuration file, or is named in the same configuration file more than once, then the duplicates will be ignored. The configuration file naming a particular provider need not be in the same jar file or other distribution unit as the provider itself. The provider must be accessible from the same class loader that was initially queried to locate the configuration file; this is not necessarily the class loader that loaded the file.

For example, an implementation of the `DateFormatProvider` class should take the form of a jar file which contains the file:

```
META-INF/services/java.text.spi.DateFormatProvider
```

And the file `java.text.spi.DateFormatProvider` should have a line such as:

```
com.foo.DateFormatProviderImpl
```

which is the fully qualified class name of the class implementing `DateFormatProvider`.

Invocation of Locale Sensitive Services

Locale sensitive factory methods and methods for name retrieval in the `java.text` and `java.util` packages invoke service provider methods when needed to support the requested locale. The methods first check whether the Java runtime environment itself supports the requested locale, and use its support if available. Otherwise, they call the `isSupportedLocale` methods of installed providers for the appropriate interface to find one that supports the requested locale. If such a provider is found, its other methods are called to obtain the requested object or name. When checking whether a locale is supported, the locale's extensions are ignored by default. (If locale's extensions should also be checked, the `isSupportedLocale` method must be overridden.) If neither the Java runtime environment itself nor an installed provider supports the requested locale, the methods go through a list of candidate locales and repeat the availability check for each until a match is found. The algorithm used for creating a list of candidate locales is same as the one used by `ResourceBundle` by default (see `getCandidateLocales` for the details). Even if a locale is resolved from the candidate list, methods that return requested objects or names are invoked with the original requested locale including `Locale` extensions. The Java runtime environment must support the root locale for all locale sensitive services in order to guarantee that this process terminates.

Providers of names (but not providers of other objects) are allowed to return null for some name requests even for locales that they claim to support by including them in their return value for `getAvailableLocales`. Similarly, the Java runtime environment itself may not have all names for all locales that it supports. This is because the sets of objects for which names are requested can be large and vary over time, so that it's not always feasible to cover them completely. If the Java

runtime environment or a provider returns null instead of a name, the lookup will proceed as described above as if the locale was not supported.

Starting from JDK8, the search order of locale sensitive services can be configured by using the "java.locale.providers" system property. This system property declares the user's preferred order for looking up the locale sensitive services separated by a comma. It is only read at the Java runtime startup, so the later call to System.setProperty() won't affect the order.

For example, if the following is specified in the property:

```
java.locale.providers=SPI,JRE
```

where "SPI" represents the locale sensitive services implemented in the installed SPI providers, and "JRE" represents the locale sensitive services in the Java Runtime Environment, the locale sensitive services in the SPI providers are looked up first.

There are two other possible locale sensitive service providers, i.e., "CLDR" which is a provider based on Unicode Consortium's CLDR Project, and "HOST" which is a provider that reflects the user's custom settings in the underlying operating system. These two providers may not be available, depending on the Java Runtime Environment implementation. Specifying "JRE,SPI" is identical to the default behavior, which is compatibile with the prior releases.

Since:

1.6

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	**LocaleServiceProvider**()	Sole constructor.

Method Summary

Abstract Methods

Concrete Methods

Modifier and Type	Method	Description
abstract **Locale**[]	**getAvailableLocales**()	Returns an array of all locales for which this

		locale service provider can provide localized objects or names.
boolean	**isSupportedLocale**(**Locale** locale)	Returns `true` if the given `locale` is supported by this locale service provider.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

LocaleServiceProvider

```
protected LocaleServiceProvider()
```

Sole constructor. (For invocation by subclass constructors, typically implicit.)

Method Detail

getAvailableLocales

```
public abstract Locale[] getAvailableLocales()
```

Returns an array of all locales for which this locale service provider can provide localized objects or names. This information is used to compose `getAvailableLocales()` values of the locale-dependent services, such as `DateFormat.getAvailableLocales()`.

The array returned by this method should not include two or more `Locale` objects only differing in their extensions.

Returns:

An array of all locales for which this locale service provider can provide localized objects or names.

isSupportedLocale

```
public boolean isSupportedLocale(Locale locale)
```

Returns `true` if the given `locale` is supported by this locale service provider. The given `locale` may contain extensions that should be taken into account for the support determination.

The default implementation returns `true` if the given `locale` is equal to any of the available `Locale`s returned by `getAvailableLocales()` with ignoring any extensions in both the given `locale` and the available locales. Concrete locale service provider implementations should override this method if those implementations are `Locale` extensions-aware. For example, `DecimalFormatSymbolsProvider` implementations will need to check extensions in the given `locale` to see if any numbering system is specified and can be supported. However, `CollatorProvider` implementations may not be affected by any particular numbering systems, and in that case, extensions for numbering systems should be ignored.

Parameters:

`locale - a Locale to be tested`

Returns:

`true if the given locale is supported by this provider; false otherwise.`

Throws:

`NullPointerException - if the given locale is null`

Since:

1.8

See Also:

```
Locale.hasExtensions(), Locale.stripExtensions()
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.